

Documento Técnico da Solução com GenAI

Arquitetura, modelos, RAG, agentes, observabilidade e n8n

Plataforma YouVisa de gestão de processos de visto integrada com agente de IA generativa capaz de tirar dúvidas, buscar informações turísticas via RAG, consultar processos do usuário e disparar fluxos de automação externa.



Sumário

1.	Descrição do problema	3
2.	Arquitetura da solução	4
3.	Modelos utilizados	6
4.	Stack tecnológica	7
5.	Aplicação dos conteúdos da disciplina ★	8
6.	Engenharia de contexto	10
7.	Observabilidade e monitoramento	11
8.	Limitações da solução	12
9.	Melhorias futuras	13
10.	Demonstração prática	14

1. Descrição do problema

1.1 Contexto

A obtenção de vistos para viagens internacionais é um processo notoriamente burocrático, fragmentado e estressante para o cidadão brasileiro. As dores recorrentes incluem:

- **Falta de transparência:** o solicitante não sabe em qual etapa o processo está, o que gera ansiedade e múltiplos contatos com call centers.
- **Documentação confusa:** cada tipo de visto (turismo, estudo, trabalho, residência) tem requisitos próprios, frequentemente desatualizados nos sites consulares.
- **Pouca personalização:** o usuário não tem orientação sobre o destino — pontos turísticos, dicas culturais, melhor época para viajar.
- **Atendimento ineficiente:** os canais tradicionais (e-mail, telefone) têm alta latência e baixa escalabilidade.

1.2 Solução proposta

A **YouVisa** é uma plataforma SaaS que centraliza toda a jornada do visto em um único produto, oferecendo:

1. Painel de acompanhamento em tempo real do processo (timeline visual, status, prazos).
2. Cadastro e gestão de múltiplas solicitações por usuário.
3. Assistente virtual "**Inho**" baseado em IA Generativa, capaz de responder dúvidas, sugerir destinos, consultar processos e disparar fluxos de automação.
4. Geração automatizada de guias turísticos personalizados em PDF.

1.3 Público-alvo

Segmento	Perfil	Necessidade principal
Primário	Brasileiros 22–50 anos	Acompanhar processo de visto turismo / estudo / trabalho
Secundário	Agências de viagem	Terceirizar processamento de vistos para múltiplos clientes

2. Arquitetura da solução

2.1 Visão geral

A solução é dividida em **quatro camadas independentes** que se comunicam por HTTP, possibilitando deploy e evolução desacoplados.

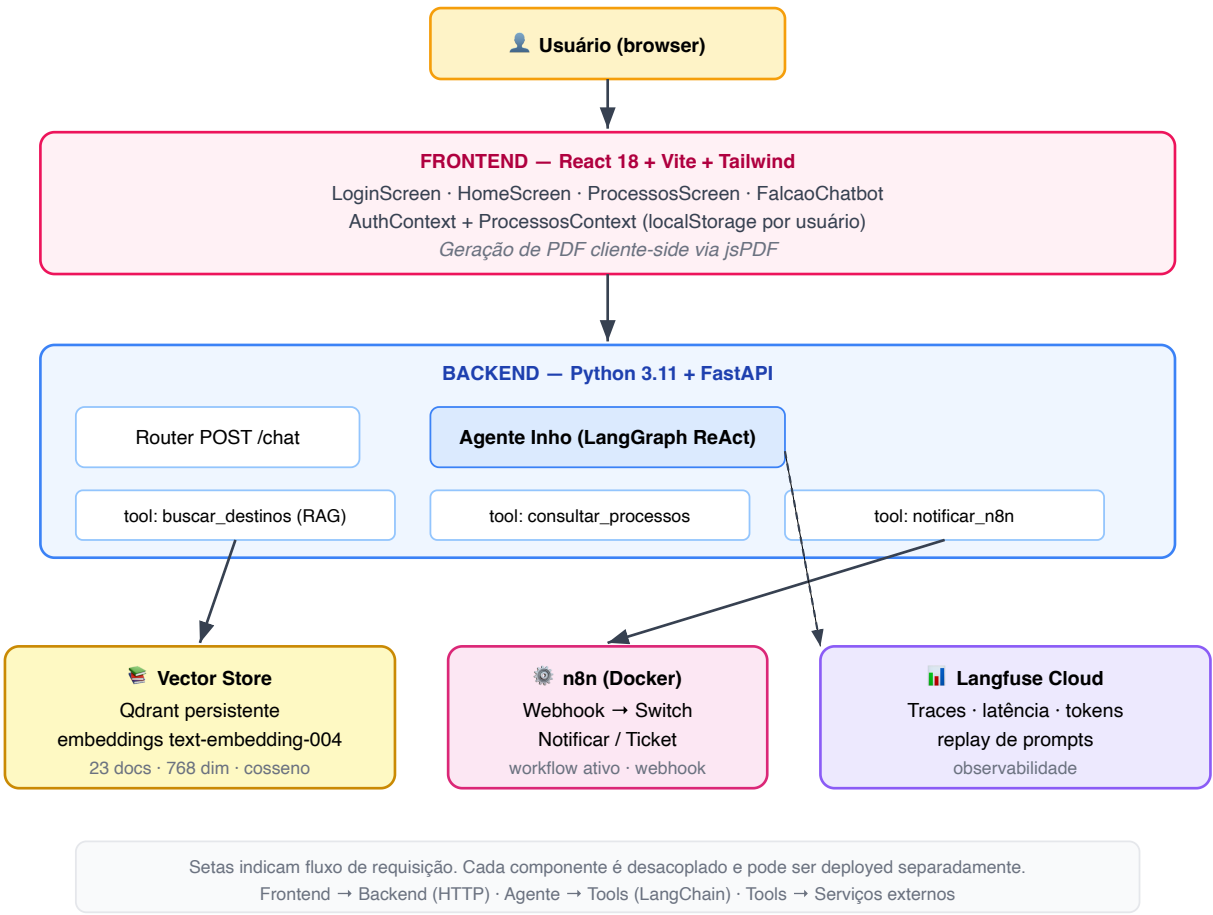


Figura 1 — Visão geral da arquitetura em 4 camadas: usuário, frontend, backend (com agente e tools) e serviços externos (vector store, n8n, observabilidade).

2.2 Camadas

Frontend (/src)

SPA React responsável pela interface, autenticação simulada e persistência de dados de usuário no `localStorage`. O componente `FalcãoChatbot` envia as mensagens do usuário ao backend e renderiza a resposta do agente, com fallback gracioso se o backend estiver offline.

Backend (/backend)

API FastAPI com um único endpoint principal (`POST /chat`) que recebe a mensagem do usuário, histórico e snapshot dos processos do user logado, invoca o agente Inho (LangGraph) e retorna a resposta final.

Agentes e Tools

Implementados com **LangGraph (ReAct)** sobre **Gemini 2.0 Flash**. O agente decide autonomamente qual tool acionar:

Tool	Função
<code>buscar_destinos</code>	Busca semântica RAG sobre 23 atrações turísticas em 5 países
<code>consultar_meus_processos</code>	Lê o snapshot dos processos do user logado
<code>notificar_evento_n8n</code>	Dispara webhook para automação no n8n

Vector Store

Qdrant persistente (`backend/data/qdrant_storage/`) com embeddings de 768 dimensões gerados pelo modelo `text-embedding-004` do Google. Métrica de similaridade: `coseno`.

Automação

n8n rodando em container Docker (`n8n/docker-compose.yml`) com workflow ativo (`workflows/youvisa-evento.json`) que recebe eventos via webhook e roteia para diferentes ações (notificar suporte, abrir ticket).

Observabilidade

Langfuse Cloud captura traces de cada interação com o agente, permitindo inspecionar latência, custo de tokens e qualidade dos prompts.

Transcrição de voz (multimodalidade)

O frontend grava o microfone do usuário com a Web API `MediaRecorder` e envia o blob de áudio ao endpoint `POST /transcrever`. O backend delega ao modelo **Whisper Large v3 Turbo** hospedado pela **Groq Cloud**, que devolve a transcrição em PT-BR. O texto então alimenta o mesmo fluxo do agente Inho, fechando o ciclo voz → texto → resposta.

2.3 Sequência de uma chamada típica

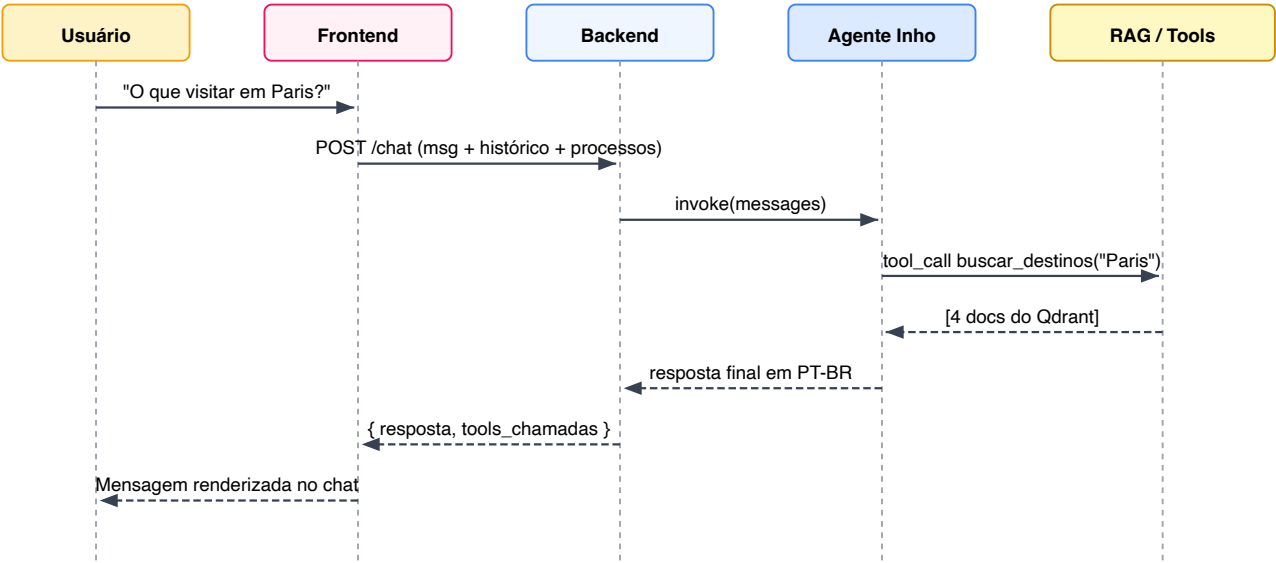


Figura 2 — Diagrama de sequência de uma chamada típica ao agente. Setas pontilhadas representam respostas.

3. Modelos utilizados

3.1 Modelo de chat — Gemini 2.0 Flash

Nome	gemini-2.0-flash-exp
Tipo	Texto generativo (LLM)
Provedor	Google AI Studio
Janela de contexto	1.000.000 tokens (1M)
Forma de acesso	API REST + SDK google-genai
Motivo da escolha	Gratuito no free tier (60 req/min), latência baixa (~600ms primeiro token), suporte nativo a tool calling, janela de contexto enorme dispensa estratégias complexas de truncamento de histórico
Limitações	Modelo experimental — pode ter mudanças de comportamento. Free tier tem rate limit. Qualidade levemente inferior ao Gemini 2.5 Pro em raciocínio complexo

3.2 Modelo de embeddings — text-embedding-004

Nome	text-embedding-004
Tipo	Embeddings densos (texto → vetor)
Provedor	Google AI Studio
Dimensões	768
Forma de acesso	API REST + SDK google-genai
Motivo da escolha	Gratuito, mesmo provedor do LLM (consistência semântica entre indexação e geração), boa performance em português, batch endpoint para indexação rápida
Limitações	768 dim já é suficiente para nossa base pequena (23 docs). Para escala maior, considerar text-embedding-3-large da OpenAI (3072 dim)

3.3 Modelo de transcrição (STT) — Whisper Large v3 Turbo

Nome	whisper-large-v3-turbo
Tipo	Speech-to-Text (áudio → texto)
Provedor	Groq Cloud
Idiomas	Multilíngue (forçado para pt em nosso uso)
Forma de acesso	API REST + SDK groq (Python)
Motivo da escolha	Inferência muito rápida (~300ms para áudios curtos graças à infraestrutura LPU da Groq), free tier generoso para demonstração, aceita formatos webm/ogg/mp3/wav que o navegador grava nativamente via MediaRecorder , qualidade de transcrição em português equivalente ao Whisper original
Limitações	Limite de tamanho do arquivo (25MB) e duração (alguns minutos por chamada). Free tier tem rate limits. Gravações com muito ruído de fundo ainda têm queda de qualidade

4. Stack tecnológica

4.1 Frontend

Ferramenta	Papel no projeto
React 18	Biblioteca de UI, gerencia estado e renderização
Vite 5	Build tool e dev server (HMR)
Tailwind CSS 3	Estilização utility-first, design system consistente
jsPDF + autotable	Geração client-side de guias turísticos em PDF
Context API	Gestão de estado global (autenticação e processos)

4.2 Backend

Ferramenta	Papel no projeto
Python 3.11	Linguagem principal do backend
FastAPI	Framework web assíncrono, validação Pydantic, OpenAPI auto-geradas
Uvicorn	ASGI server de produção
Pydantic 2	Schemas tipados das requisições e respostas
python-dotenv	Carregamento de variáveis de ambiente

4.3 GenAI

Ferramenta	Papel no projeto
google-genai	SDK oficial Google para Gemini (chat e embeddings)
groq	SDK oficial Groq para Whisper Large v3 Turbo (transcrição de áudio)
LangGraph	Orquestração do agente ReAct com tools
LangChain Core	Abstrações de tools (@tool) e mensagens
langchain-google-genai	Adapter LangChain ↔ Gemini
Qdrant	Vector store persistente (modo local em arquivos ou Qdrant Cloud)
MediaRecorder API	Captura de áudio do microfone do usuário no navegador (Web API nativa)

4.4 Automação e Observabilidade

Ferramenta	Papel no projeto
n8n (Docker)	Workflow de automação para eventos disparados pelo agente
Langfuse	Observabilidade de LLMs — traces, métricas, custos

4.5 Infraestrutura

Ferramenta	Papel no projeto
Docker Compose	Orquestração local do n8n
Git + GitHub	Versionamento e repositório remoto

5. Aplicação dos conteúdos da disciplina

Esta é a seção de maior peso na avaliação. Detalhamos abaixo, item por item, ONDE cada tema da ementa foi aplicado no projeto, com referências diretas a arquivos e linhas.

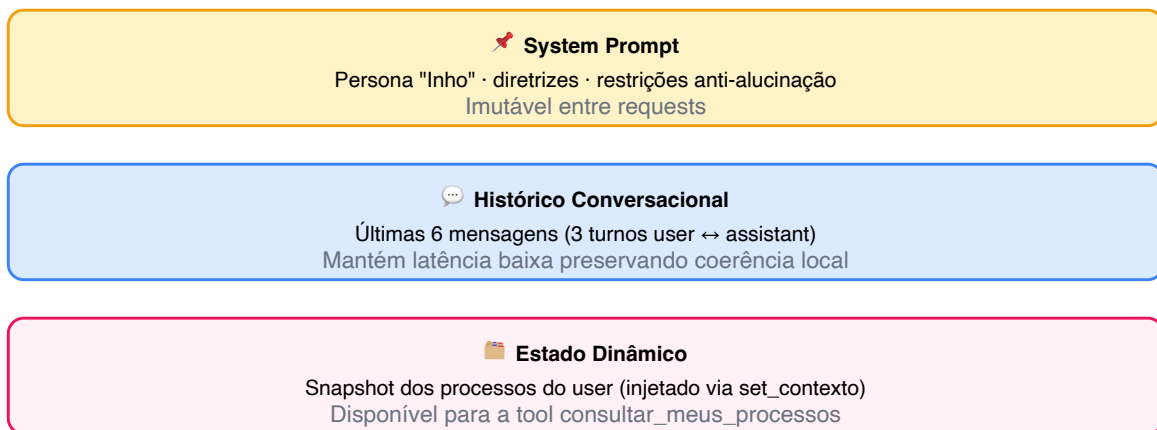
Tema da disciplina	Aplicação concreta no projeto
Prompt Engineering	System prompt estruturado em <code>backend/app/agents/inho.py</code> com persona definida (Inho, falcão mascote), papel, diretrizes de tom, restrições contra alucinação ("baseie a resposta nos dados retornados — não invente atrações"), formato de saída e uso controlado de emojis.
RAG	Pipeline completo em <code>backend/app/rag/store.py</code> : (1) chunking por destino, (2) embeddings via <code>text-embedding-004</code> , (3) indexação em Qdrant persistente, (4) recuperação top-k=4 com filtro opcional por país. A tool <code>buscar_destinos</code> é chamada pelo agente quando o user pergunta sobre pontos turísticos.
Agentes	Agente ReAct em <code>backend/app/agents/inho.py</code> usando <code>langgraph.prebuilt.create_react_agent</code> . O agente decide autonomamente quando chamar cada tool, realizando múltiplos turnos de raciocínio se necessário.
MCP / Integração de ferramentas	Três tools registradas via decorator <code>@tool</code> do LangChain Core (<code>backend/app/tools/</code>): <code>buscar_destinos</code> (RAG), <code>consultar_meus_processos</code> (estado da aplicação), <code>notificar_evento_n8n</code> (integração externa via webhook). O agente recebe schemas tipados e o LLM faz function calling estruturado.
Observabilidade	Langfuse integrado em <code>backend/app/observability.py</code> via context manager <code>trace()</code> . Cada chamada ao endpoint <code>/chat</code> cria um trace com <code>user_id</code> , tamanho da mensagem e número de processos do usuário. Dashboard em <code>cloud.langfuse.com</code> mostra latência, tokens, custo e replay de prompts.
Deploy	Backend preparado para deploy em Render (Python free tier), frontend em Vercel ou Netlify. Variáveis sensíveis isoladas em <code>.env</code> . CORS configurado por ambiente. n8n em container Docker portátil.
Multimodalidade	Entrada por voz: o usuário grava áudio diretamente no chatbot pelo botão de microfone (<code>MediaRecorder</code> API). O blob é enviado ao endpoint <code>POST /transcrever</code> , que usa o Groq Whisper Large v3 Turbo para transcrever em PT-BR; o texto resultante é injetado no input para o agente processar. Saída multimodal: o agente coordena com o frontend para gerar PDFs ricos via jsPDF (textos + estrutura visual + capa).
Engenharia de contexto	(i) Construção do prompt dinâmico no router <code>/chat</code> : histórico das últimas 6 mensagens + system prompt + injeção do snapshot de processos via <code>set_contexto</code> . (ii) Chunking semântico por documento (granularidade ideal para a base atual). (iii) Recuperação top-k filtrada para reduzir ruído.
Tool calling estruturado	Os argumentos das tools são tipados (<code>query: str, pais: str None</code>). O LLM emite chamadas em JSON validadas pelo LangChain antes da execução.
Memória do agente	Memória de curto prazo: histórico das últimas 6 mensagens passado a cada turno. Memória de longo prazo: estado dos processos do user persistido no <code>localStorage</code> do front, enviado a cada requisição.
Fine-tuning	Não aplicado nesta versão — o <code>text-embedding-004</code> e o <code>gemini-2.0-flash</code> zero-shot já dão resultados suficientes para o domínio. Documentado como melhoria futura na seção 9 caso a base cresça acima de 1000 documentos.

6. Engenharia de contexto

6.1 Como o contexto é montado

A cada requisição, o router `backend/app/routers/chat.py` constrói o contexto do agente em três camadas:

1. **System prompt** (`SYSTEM_PROMPT` em `inho.py`) — define persona, papel, restrições e estilo. Imutável entre requests.
2. **Histórico conversacional** — últimas 6 mensagens (3 turnos user↔assistant) extraídas do front. Truncamento mantém latência baixa sem perder coerência local.
3. **Estado dinâmico** — snapshot dos processos do user injetado via `set_contexto(...)` antes do `agente.invoke(...)`, disponível para a tool `consultar_meus_processos`.



Camadas combinadas ≈ 600–1500 tokens, bem dentro do 1M de janela do Gemini.

Figura 3 — Camadas de construção de contexto a cada requisição ao agente.

6.2 Chunking

A base de conhecimento (`backend/data/knowledge_base.json`) é estruturada **uma atração por documento** (23 docs). Cada chunk é montado em `_chunk_para_texto()` concatenando os campos `atracao`, `cidade`, `pais`, `categoria`, `descricao` e `dica` em um texto coerente e auto-contido.

Por que não chunking automático por tamanho? Os documentos têm tamanho similar e baixo (média ~200 caracteres), e o significado está fortemente acoplado à entidade "atração". Chunking por janela deslizante quebraria a relação semântica.

6.3 Embeddings

Geração em batch via `models.embed_content` do Gemini, modelo `text-embedding-004`, dimensão 768. Indexação é **idempotente**: se a contagem na coleção já bate com o JSON, pula a etapa.

6.4 Recuperação

```
store.buscar(query, k=4, pais=None) :
```

1. Embed da query com o **mesmo modelo** da indexação (consistência semântica).

2. `client.query_points` no Qdrant retorna top-k por similaridade de cosseno.
3. Filtro opcional via `where={"pais": pais}` quando o agente já sabe a restrição geográfica.
4. Resultado retornado como string formatada `[1] ... [2] ...` para o LLM citar fonte.

6.5 Memória do agente

- **Curto prazo:** histórico recente passado a cada turno (mantém coerência conversacional).
- **Longo prazo (estado da aplicação):** processos do user no localStorage do front. Não persistimos histórico de chat no backend para simplificar e preservar privacidade.

7. Observabilidade e monitoramento

7.1 Logs

- **Backend:** logs estruturados em pontos críticos (indexação RAG, falhas no n8n). Em produção, plugar `structlog` para JSON logs.
- **n8n:** cada execução é registrada na própria UI do n8n (histórico de execuções, payloads, tempos por nó).
- **Frontend:** erros de fetch caem em fallback gracioso (modo legado rule-based) — registrados em `console.error`.

7.2 Métricas (Langfuse)

Métrica	Descrição
Latência	Por chamada de LLM e por trace completo
Tokens	Input/output e custo estimado por modelo
Tools chamadas	Quais tools o agente acionou em cada interação
Taxa de erro	Por tool e por trace, com stack trace
Replay	Reproduzir qualquer interação do passado para debugging

7.3 Como falhas são monitoradas

- Erros no agente são capturados pelo `try/except` em `chat.py` e retornados como HTTP 500 com mensagem.
- Cada exception também é registrada no trace ativo do Langfuse, permitindo identificar padrões.
- Webhook do n8n tem timeout de 5s — falha não derruba o agente, apenas retorna mensagem informativa.
- O endpoint `/health` expõe status dos componentes (Langfuse ativo, n8n configurado) para health checks externos.

8. Limitações da solução

Limitação	Impacto e mitigação atual
Alucinação	Mitigada parcialmente pelo system prompt e pelo RAG. Ainda assim, o LLM pode confabular informações fora do escopo (ex: preços, horários).
Latência	1.5–3 segundos por resposta (Gemini 2.0 Flash, sem streaming). Inaceitável para uso em massa em pico sem caching ou modelo local.
Custo	Free tier cobre desenvolvimento e demo. Em produção (>60 req/min), passa para tier pago — estimativa \$0.0003/req com nossa janela de contexto.
Contexto limitado	Histórico truncado em 6 mensagens. Conversas longas perdem coerência inicial. Solução: implementar memória resumida (summary buffer).
Modelo experimental	<code>gemini-2.0-flash-exp</code> pode ter mudanças não documentadas. Free tier tem rate limit (60 req/min).
Sem streaming	Resposta só após o agente terminar todos os turnos. UX poderia ser melhor com streaming token-a-token.
Base de conhecimento pequena	23 atrações em 5 países. Para uso real, expandir para milhares exigiria revisão da estratégia de chunking e vector DB gerenciado.
Segurança	Autenticação simulada com mock no frontend. Sem backend de auth real — adequado apenas para protótipo acadêmico.

9. Melhorias futuras

Melhoria	Descrição
Multi-agent	Especializar em sub-agentes (Documentação, Financeiro, Turismo) coordenados por um Supervisor (CrewAI ou LangGraph com <code>Send</code>).
Voz	Input multimodal via Web Speech API no browser (zero custo) ou Whisper local. Output via TTS (ex: ElevenLabs).
Modelos locais	Fallback para Llama 3.2 ou Phi-3 via Ollama em modo offline ou para queries simples.
Fine-tuning	Treinar embeddings customizados sobre o vocabulário de vistos brasileiros (ex: "DS-160", "ESTA") quando a base passar de 1000 docs.
Streaming	Trocar <code>agente.invoke</code> por <code>agente.astream_events</code> e SSE no FastAPI para resposta token-a-token.
Guardrails	Adicionar <code>guardrails-ai</code> ou validação Pydantic estrita de output.
Auth real	Substituir mock por Firebase Auth ou Supabase.
Cache de embeddings	Cache LRU em memória de queries frequentes para reduzir latência e custo.
Automação n8n expandida	Workflows para envio de email com PDF gerado, lembretes de prazo, integração com Google Calendar.
Internacionalização	Suporte a inglês e espanhol (público estrangeiro vivendo no Brasil).

10. Demonstração prática

10.1 Como subir o ambiente completo

```
# 1) Frontend
npm install
npm run dev          # http://localhost:5173

# 2) Backend
cd backend
python -m venv .venv && source .venv/bin/activate
pip install -r requirements.txt
cp .env.example .env # preencher GOOGLE_API_KEY
uvicorn app.main:app --reload --port 8000

# 3) n8n (em outro terminal)
cd n8n
docker compose up -d
# importar workflows/youvisa-evento.json em http://localhost:5678
```

10.2 Fluxo de uso

1. Usuário faz login (joao@youvisa.com / 123456)
2. Cria solicitação de visto (Tipo: Turismo, Status: Em análise)
3. Abre o chatbot Inho (canto inferior direito)
4. Pergunta: *"Quais pontos turísticos eu vejo em Paris?"*
5. Agente raciocina:
 - Decide chamar `buscar_destinos(query="Paris", pais="França")`
 - Recebe 4 destinos relevantes do Qdrant
 - Sintetiza resposta em linguagem natural
6. Resposta aparece no chat (~2s)
7. Trace completo é registrado no Langfuse

10.3 Exemplo de trace observado no Langfuse

```
Trace: chat_inho
├─ user_id: u-001
├─ n_processos: 2
├─ Span 1: gemini-2.0-flash-exp
│   ├── input: 612 tokens
│   ├── output: 89 tokens (tool_call buscar_destinos)
│   └─ latência: 720ms
├─ Span 2: tool buscar_destinos
│   ├── query: "Paris"
│   ├── resultados: 4 docs
│   └─ latência: 230ms (embed + qdrant query)
├─ Span 3: gemini-2.0-flash-exp
│   ├── input: 1.247 tokens
│   ├── output: 178 tokens
│   └─ latência: 1.1s
└─ Latência total: 2.05s
```

10.4 Repositório

Código completo, organizado por camadas, em: https://github.com/ViniciusOnii/you_front

```
you_front/
├── src/                # Frontend React
├── backend/            # Backend Python + agente
│   ├── app/
│   │   ├── agents/    # LangGraph
│   │   ├── rag/        # Qdrant
│   │   ├── tools/      # Tool registrations
│   │   └── routers/
│   └── data/          # Knowledge base
├── n8n/                # Docker compose + workflow
└── docs/               # Este documento
```